OTEL-99: Best Practice Summary

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/28/2026

Introduction

This notebook consolidates every actionable best practice from the OTEL series (notebooks 01-08) into definitive, reference-ready tables. Each practice specifies the exact setting or value to use, a priority level, and its category. Use this as a checklist when planning, deploying, or auditing OpenTelemetry integration with Dynatrace.

Table of Contents

1. Collector Configuration
2. Deployment and Sizing
3. Security
4. Dynatrace OTLP Integration
5. Resource Attributes and Entity Mapping
6. Trace Instrumentation
7. Metrics Instrumentation
8. Logs Instrumentation
9. Semantic Conventions
10. Performance and Reliability
11. Troubleshooting and Observability

---

1. Collector Configuration

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Always use the `batch` processor	`processors.batch.timeout: 10s`, `send_batch_size: 1000`, `send_batch_max_size: 1500`	Critical	OTEL-02
2	Always use the `memory_limiter` processor	`processors.memory_limiter.check_interval: 1s`, `limit_mib:` 80% of container memory, `spike_limit_mib:` 25% of limit	Critical	OTEL-02
3	Place `memory_limiter` first in every pipeline	`processors: [memory_limiter, resource, batch]`	Critical	OTEL-02, OTEL-08
4	Pin Collector image to a specific version	`image: otel/opentelemetry-collector-contrib:0.151.0` (never `latest`)	Critical	OTEL-03, OTEL-07
5	Validate config before deploy	Run `otelcol-contrib validate --config=config.yaml`	Critical	OTEL-08

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Use the Contrib or Dynatrace distribution	`otel/opentelemetry-collector-contrib` or `ghcr.io/dynatrace/dynatrace-otel-collector/dynatrace-otel-collector`	Recommended	OTEL-02, OTEL-07
7	Use `GOMEMLIMIT` instead of deprecated `memory_ballast`	Set `GOMEMLIMIT` env var to ~80% of container memory limit	Critical	OTEL-02
8	Use environment variables for all secrets	`${DT_API_TOKEN}`, `${DT_ENV_ID}` in YAML config	Critical	OTEL-02
9	Add a default `service.name` via resource processor	`processors.resource.attributes: [{key: service.name, value: "unknown-service", action: insert}]`	Recommended	OTEL-07, OTEL-08
10	Enable retry on failure for exporters	`retry_on_failure.enabled: true`, `initial_interval: 5s`, `max_interval: 30s`, `max_elapsed_time: 300s`	Recommended	OTEL-03, OTEL-07

2. Deployment and Sizing

#	Best Practice	Recommended Setting/Value	Priority	Source
11	Use DaemonSet (agent mode) for per-node collection	`kind: DaemonSet` in K8s manifest	Recommended	OTEL-03
12	Use Deployment (gateway mode) for centralized processing	`kind: Deployment` with `replicas: 2+`	Recommended	OTEL-03
13	Size gateway resources by measured throughput	Community starting points — <1k spans/s: 100m/256Mi; <10k: 500m/1Gi; <100k: 2/4Gi; >100k: 4+/8Gi+ — then resize from your own memory headroom	Critical	OTEL-03
14	Set `memory_limiter.limit_mib` to 80% of container memory limit	Container limit 1Gi -> `limit_mib: 800`	Critical	OTEL-03
15	Deploy gateway with minimum 2 replicas for HA	`spec.replicas: 2`	Recommended	OTEL-03
16	Add PodDisruptionBudget for gateway	`minAvailable: 1`	Recommended	OTEL-03
17	Use HorizontalPodAutoscaler at 70% CPU target	`averageUtilization: 70`, `minReplicas: 2`, `maxReplicas: 10`	Recommended	OTEL-03
18	Enable persistent queue for gateway exporters	`sending_queue.enabled: true`, `queue_size: 1000`, `storage: file_storage`	Recommended	OTEL-03
19	Install via Helm for K8s deployments	`helm install otel-collector open-telemetry/opentelemetry-collector --set mode=deployment --set image.repository=ghcr.io/open-telemetry/opentelemetry-collector-releases/opentelemetry-collector-k8s --set command.name=otelcol-k8s` (the chart has no default image)	Recommended	OTEL-03

#	Best Practice	Recommended Setting/Value	Priority	Source
20	Fix time zones where text timestamps are parsed	OTLP timestamps are epoch nanoseconds with no zone; set the location on the `file_log` time parser, not the container `TZ`	Recommended	OTEL-08

3. Security

#	Best Practice	Recommended Setting/Value	Priority	Source
21	Store API tokens in Kubernetes Secrets	`kind: Secret`, reference via `secretKeyRef`	Critical	OTEL-03
22	Enable TLS on receiver endpoints	`tls.cert_file`, `tls.key_file` on `otlp.protocols.grpc` and `.http`	Critical	OTEL-03
23	Enable mTLS for internal traffic	`tls.client_ca_file` on receiver	Optional	OTEL-03
24	Apply NetworkPolicy to restrict Collector ingress/egress	Allow ingress 4317/4318 from app namespaces; egress 443 to Dynatrace only	Recommended	OTEL-03
25	Strip sensitive attributes in the processor pipeline	`processors.attributes.actions: [{key: api.key, action: delete}]`	Critical	OTEL-02
26	Never embed tokens in code or config files	Always use env vars or Secret references	Critical	OTEL-07
27	Never include PII in span or metric attributes	Exclude user emails, SSNs, credit card numbers	Critical	OTEL-04

4. Dynatrace OTLP Integration

#	Best Practice	Recommended Setting/Value	Priority	Source
28	Use OTLP/HTTP with binary protobuf for direct Dynatrace ingest	`exporters.otlphttp.endpoint: https://{env-id}.live.dynatrace.com/api/v2/otlp` — JSON payloads are not supported	Critical	OTEL-01, OTEL-07
29	Route gRPC through a Collector (gRPC is not supported for direct DT ingest)	Collector with `otlp` gRPC receiver + `otlphttp` exporter to Dynatrace	Critical	OTEL-01, OTEL-07
30	Match the auth scheme to the token	Classic token: `Authorization: Api-Token <token>`. Platform token: `Authorization: Bearer <token>`	Critical	OTEL-07
31	Create a dedicated token with minimum required scopes	Platform: `openpipeline:traces:ingest` + `openpipeline:metrics:ingest` + `openpipeline:logs:ingest`. Classic: `openTelemetryTrace.ingest` + `metrics.ingest` + `logs.ingest`	Critical	OTEL-01, OTEL-07
32	Use Dynatrace Collector distribution for production	`ghcr.io/dynatrace/dynatrace-otel-collector/dynatrace-otel-collector`	Recommended	OTEL-07
33	Use ActiveGate endpoint for on-prem or network-restricted envs	`https://{activegate-host}:9999/e/{env-id}/api/v2/otlp`	Recommended	OTEL-07

#	Best Practice	Recommended Setting/Value	Priority	Source
34	Use Dynatrace Operator OTLP auto-config for K8s (v1.8+)	Annotation `otlp-exporter-configuration.dynatrace.com/inject` for per-pod control	Recommended	OTEL-07
35	Inventory OTLP metric dimensions before enabling Advanced OTLP metric dimensions	All attributes become dimensions, keys keep their case, and histograms change type (Counter → Histogram) — re-check SLOs, alerts and dashboards afterwards	Critical	OTEL-05, OTEL-07
36	Treat OTel and OneAgent as complementary — decide per workload	OTel-only is a complete path; OneAgent where fully managed collection or runtime depth is wanted; both where that fits. Mixed estates are normal	Critical	OTEL-01
37	Keep existing OTel instrumentation when adding OneAgent	Layer, don't convert; pick one ingestion path per process (Span Sensor **or** OTLP export) to avoid duplicate spans	Critical	OTEL-01, OTEL-07
38	Add OneAgent where runtime-access capabilities are needed	Application Security (RVA / RAP), Live Debugger (Java, Node.js; OneAgent 1.309+), automatic RUM JavaScript injection — not provided by OTel telemetry	Recommended	OTEL-01
39	Manage large Collector fleets centrally	Bindplane (OpAMP-based fleet management, multi-destination routing) — optional, not a prerequisite; never write into the reserved `dt.bindplane.*` namespace (SaaS 1.346)	Recommended	OTEL-07
40	Give OTel-only hosts host and process entities	Collector `hostmetrics` with the Dynatrace reference configuration + activate the OpenTelemetry Host Monitoring extension from Hub	Recommended	OTEL-07

5. Resource Attributes and Entity Mapping

#	Best Practice	Recommended Setting/Value	Priority	Source
41	Always set `service.name` resource attribute	`resource.create({"service.name": "checkout-api"})`	Critical	OTEL-07, OTEL-08
42	Set `service.version` for release tracking	`"service.version": "1.2.3"`	Recommended	OTEL-07
43	Set `service.namespace` for service grouping	`"service.namespace": "ecommerce"`	Recommended	OTEL-07
44	Set `deployment.environment.name` for environment identification	`"deployment.environment.name": "production"` (use stable name, not legacy `deployment.environment`)	Recommended	OTEL-01, OTEL-07
45	Set K8s resource attributes for entity linking	`k8s.namespace.name`, `k8s.pod.name`, `k8s.deployment.name`	Recommended	OTEL-01, OTEL-07
46	Use `resourcedetection` processor for cloud/K8s metadata	`processors.resourcedetection` with env, gcp, aws, azure, or k8s detectors	Recommended	OTEL-02

6. Trace Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Source
47	Start with auto-instrumentation, add manual spans for business logic	Auto-instrument frameworks; add custom spans with `tracer.start_as_current_span()`	Recommended	OTEL-04
48	Use low-cardinality span names	`HTTP GET /users/{id}` not `HTTP GET /users/12345`	Critical	OTEL-04
49	Use descriptive, operation-based span names	`process_order`, `db.query` not `order_abc123`, `SELECT * FROM...`	Critical	OTEL-04
50	Always record errors on spans	`span.record_exception(e)` + `span.set_status(StatusCode.ERROR, str(e))`	Critical	OTEL-04
51	Add business context as span attributes	`span.set_attribute("order.id", order_id)`	Recommended	OTEL-04
52	Use span events for key milestones	`span.add_event("payment_completed", {"transaction_id": txn_id})`	Optional	OTEL-04
53	Use `BatchSpanProcessor` (never `SimpleSpanProcessor` in production)	`provider.add_span_processor(BatchSpanProcessor(exporter))`	Critical	OTEL-04
54	Use W3C Trace Context propagation (required for OneAgent interop)	`W3CTraceparentPropagator` + `W3CTracestatePropagator`	Critical	OTEL-04, OTEL-07
55	Propagate context across async boundaries	In Python, asyncio tasks copy the current context when created — no explicit attach/detach is needed around `await` / `gather` / `create_task`	Critical	OTEL-04, OTEL-08
56	Propagate context in messaging (Kafka, RabbitMQ)	Inject context into message headers on produce; extract on consume	Recommended	OTEL-04
57	Use `TraceIdRatioBased` sampler to control trace volume	`TraceIdRatioBased(0.1)` for 10% sampling in high-throughput envs	Recommended	OTEL-08

7. Metrics Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Source
58	Choose the correct instrument type	Counter for monotonic; Histogram for durations/sizes; UpDownCounter for gauges; ObservableGauge for current state	Critical	OTEL-05
59	Use hierarchical metric naming	`<domain>.<component>.<metric>` e.g. `http.server.request.duration`	Recommended	OTEL-05

#	Best Practice	Recommended Setting/Value	Priority	Source
60	Use the semantic-convention unit	`http.server.request.duration` is a Histogram in `s` (seconds); `unit="By"` for bytes. Never reuse a convention name with a different unit	Recommended	OTEL-05
61	Keep attribute cardinality low	Use route templates (`/users/{id}`), status classes (`2xx`), never user IDs or timestamps	Critical	OTEL-05
62	Configure explicit histogram bucket boundaries	`boundaries: [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10]` (seconds) for response times	Recommended	OTEL-05
63	Use Views to drop noisy internal metrics	`View(instrument_name="internal.debug.*", aggregation=DropAggregation())`	Optional	OTEL-05
64	Use `ObservableGauge` / `ObservableCounter` for system stats	Avoid polling in the request hot path	Recommended	OTEL-05
65	Set appropriate export interval	`PeriodicExportingMetricReader(exporter, export_interval_millis=60000)` (60s default)	Recommended	OTEL-05
66	Measure RED metrics for services	Rate (Counter), Errors (Counter), Duration (Histogram)	Critical	OTEL-05
67	Measure USE metrics for resources	Utilization, Saturation, Errors	Recommended	OTEL-05

8. Logs Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Source
68	Use the Log Bridge pattern (do not replace your logging library)	Attach `LoggingHandler` to standard Python logger; use Log4j/Logback appenders for Java	Critical	OTEL-06
69	Enable automatic log-trace correlation	Use OTel tracing + log bridge together; `trace_id` and `span_id` are injected automatically	Critical	OTEL-06
70	Use structured JSON logging	Output logs as JSON objects with `timestamp` , `level` , `message` , and structured fields	Recommended	OTEL-06
71	Filter DEBUG/TRACE logs at the Collector, not in the app	`processors.filter.logs.log_record: ['severity_number > 0 and severity_number < 9']` (0 = no severity; keep it)	Recommended	OTEL-06
72	Use `BatchLogRecordProcessor` for log export	Never use synchronous log export in production	Critical	OTEL-06
73	Use the file log receiver for container/file-based log collection	`receivers.file_log.include: ["/var/log/containers/*.log"]` — `file_log` from collector-contrib v0.149.0; older Collectors only know `filelog`	Recommended	OTEL-02, OTEL-06
74	Set log severity appropriately	ERROR: unexpected failures; WARN: recoverable issues; INFO: business events; DEBUG: dev only	Recommended	OTEL-06
75	Never log PII or secrets	Exclude passwords, tokens, personal data from log bodies and attributes	Critical	OTEL-06

9. Semantic Conventions

#	Best Practice	Recommended Setting/Value	Priority	Source
76	Migrate to stable HTTP semantic conventions	Use `http.request.method` (not `http.method`), `http.response.status_code` (not `http.status_code`), `url.full` (not `http.url`)	Recommended	OTEL-01
77	Migrate to stable DB semantic conventions	Use `db.namespace` (not `db.name`), `db.query.text` (not `db.statement`), `db.operation.name` (not `db.operation`)	Recommended	OTEL-01
78	Use `OTEL_SEMCONV_STABILITY_OPT_IN` to control migration	`http` for stable-only, `http/dup` for dual-emit during migration; same pattern for `database`	Recommended	OTEL-01
79	Use stable resource attribute `deployment.environment.name`	Not the legacy `deployment.environment`	Recommended	OTEL-01
80	Follow semantic conventions for all standard attributes	Use `service.name`, `service.version`, `k8s.namespace.name`, `k8s.pod.name`, etc.	Critical	OTEL-01

10. Performance and Reliability

#	Best Practice	Recommended Setting/Value	Priority	Source
81	Derive RED metrics from traces before sampling	`span_metrics` connector (`spanmetrics` before contrib v0.151.0) with histogram buckets `[100ms, 500ms, 1s, 5s]`	Optional	OTEL-02, OTEL-04
82	Filter health-check spans at the Collector	`processors.filter.traces.span: ['attributes["url.path"] == "/health"']`	Recommended	OTEL-02
83	Use fan-out to send data to multiple backends	Define multiple named exporters under `exporters` and list all in pipeline	Optional	OTEL-02
84	Increase `sending_queue.num_consumers` for high throughput	`num_consumers: 10`	Recommended	OTEL-08
85	Do not over-instrument	Only create spans for meaningful operations; avoid spans on every function call	Recommended	OTEL-04
86	Limit attribute value sizes	Avoid multi-KB strings in span attributes or log bodies	Recommended	OTEL-04
87	Configure proxy for network-restricted environments	`exporters.otlphttp.proxy_url: http://proxy.example.com:8080` or `HTTPS_PROXY` env var	Optional	OTEL-08

11. Troubleshooting and Observability

#	Best Practice	Recommended Setting/Value	Priority	Source
88	Enable `health_check` extension	`extensions.health_check.endpoint: 0.0.0.0:13133`	Critical	OTEL-02, OTEL-08

#	Best Practice	Recommended Setting/Value	Priority	Source
89	Enable `zpages` extension in non-production	`extensions.zpages.endpoint: 0.0.0.0:55679`	Recommended	OTEL-02, OTEL-08
90	Use `debug` exporter during development	`exporters.debug.verbosity: detailed` alongside production exporter	Recommended	OTEL-02, OTEL-08
91	Set Collector log level to `info` in production, `debug` for troubleshooting	`service.telemetry.logs.level: info` (default), switch to `debug` when investigating	Recommended	OTEL-08
92	Verify tracer provider is not NoOp	In Python: `print(trace.get_tracer_provider())` must not show `NoOpTracerProvider`	Critical	OTEL-08
93	Test connectivity with curl before deploying	`curl -v https://{env}.live.dynatrace.com/api/v2/otlp/v1/traces -X POST -H "Authorization: Api-Token <token>" -H "Content-Type: application/x-protobuf" --data-binary ''`	Recommended	OTEL-08
94	Troubleshoot systematically: SDK -> Collector -> Backend	Follow the data path; isolate which component is failing	Recommended	OTEL-08
95	Enable `pprof` extension for Go heap profiling	`extensions.pprof.endpoint: 0.0.0.0:1777`	Optional	OTEL-08

Summary

This notebook contains **95 best practices** extracted from the OTEL series, organized into 11 categories:

Category	Count	Critical	Recommended	Optional
Collector Configuration	10	7	3	0
Deployment and Sizing	10	2	8	0
Security	7	5	1	1
Dynatrace OTLP Integration	13	7	6	0
Resource Attributes and Entity Mapping	6	1	5	0
Trace Instrumentation	11	6	4	1
Metrics Instrumentation	10	3	6	1
Logs Instrumentation	8	4	4	0
Semantic Conventions	5	1	4	0
Performance and Reliability	7	0	4	3
Troubleshooting and Observability	8	2	5	1

Category	Count	Critical	Recommended	Optional
Total	**95**	**38**	**50**	**7**

Priority guide:
- **Critical** -- Implement before going to production. Skipping causes data loss, security exposure, or broken entity mapping.
- **Recommended** -- Implement for production-grade deployments. Improves reliability, performance, and maintainability.
- **Optional** -- Implement when the specific use case applies. Adds value in targeted scenarios.

References

- Dynatrace OpenTelemetry Integration
- Dynatrace Collector Distribution
- OTLP API Endpoints
- Configure OTLP Metrics
- OpenTelemetry Official Docs
- OTel Collector Configuration
- Semantic Conventions

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.